

數位影像處理 HW1

資訊三乙 D1249326 陳卓妍

一、原圖



(920 736 3)

二、彩色轉黑白

- 程式碼

```
import cv2
import numpy as np

img = cv2.imread('cat.jpg')

#照片轉黑白
gray = np.zeros(img.shape[:2], dtype=np.uint8) # 建立一個空的黑白影像

for y in range(img.shape[0]): # 高
    for x in range(img.shape[1]): # 寬
        b, g, r = img[y, x] # 取得原始影像的藍綠紅三個頻道值
        gray_value = int(0.114 * b + 0.587 * g + 0.299 * r) # 計算灰階值
        gray[y, x] = gray_value # 將灰階值賦值給黑白影像

cv2.imshow('Gray Image', gray)
cv2.imwrite('gray.bmp', gray) #儲存黑白影像
cv2.waitKey()
```

- 執行結果



- 心得討論

需要先建立一個空白圖檔來儲存轉成黑白的元素，且須掌握原始檔的藍綠紅，再透過計算灰階值的公式和for迴圈將每一個點複製到新圖檔中。這樣在圖檔過大時會執行相較於內建函式慢。

三、最近鄰放大/縮小

- 程式碼

```
import cv2
import numpy as np
img = cv2.imread('cat.jpg')

# 放大/縮小
h, w, c = img.shape
print(h, w, c)
scale = float(input("輸入放大 / 縮小 幾倍 (0.1~2): "))

new_h = int(h * scale) # 計算新高度
new_w = int(w * scale) # 計算新寬度

#最近鄰
nearest = np.zeros((new_h, new_w, c), dtype=img.dtype)

for y in range(new_h):
    for x in range(new_w):
        src_y = int(y / scale) # 計算對應的原始影像座標
        src_x = int(x / scale)

        #保護邊界
        src_x = min(src_x, w - 1)
        src_y = min(src_y, h - 1)
        nearest[y, x] = img[src_y, src_x]

cv2.imshow('nearest Image', nearest)
cv2.imwrite('nearest.jpg', nearest) #最近鄰
cv2.waitKey()
```

- 執行結果（縮小 0.5 倍）



(460 368 3)

- 心得討論

與內建函式相比，速度的確較慢，且for迴圈造成效率也較差。但是這也比較好理解最近鄰放大縮小的原始概念。另外與線性放大縮小相比，最近鄰顯示的圖檔畫質較差，放大時具有鋸齒狀，縮小則是會導致影像丟失機率發生。

四、線性放大/縮小

- 程式碼

```
import cv2
import numpy as np
import math

img = cv2.imread('cat.jpg')

# 放大/縮小
h, w, c = img.shape
print(h, w)
scale = float(input("輸入放大或縮小幾倍 (0.1~2): "))

new_h = int(h * scale)
new_w = int(w * scale)
print(new_h, new_w)

bilinear = np.zeros((new_h, new_w, c), dtype=img.dtype) # 建立新影像

# 雙重迴圈遍歷新影像的每個像素
for y in range(new_h):
    for x in range(new_w):
        src_x = x / scale
        src_y = y / scale

        #取得左上角的像素座標
        y0 = int(src_y)
        x0 = int(src_x)
        # 取得右下角的像素座標
        y1 = min(y0 + 1, h - 1)
        x1 = min(x0 + 1, w - 1)

        # 計算插值權重
        dy = src_y - y0
        dx = src_x - x0

        # 取得四個鄰近像素的值
        Q11 = img[y0, x0]
        Q21 = img[y0, x1]
        Q12 = img[y1, x0]
        Q22 = img[y1, x1]

        # 執行雙線性插值
        top = (1 - dx) * Q11 + dx * Q21
        bottom = (1 - dx) * Q12 + dx * Q22
        pixel = (1 - dy) * top + dy * bottom

        bilinear[y, x] = np.clip(pixel, 0, 255) #確保像素值在有效範圍內

cv2.imshow('bilinear Image', bilinear)
cv2.imwrite('bilinear.jpg', bilinear) #旋轉
cv2.waitKey()
```

- 執行結果（放大1.5倍）



(1380 1104 3)

- 心得討論

需要手動調整座標位置，還須保護邊界。另外，還需手動計算權重，並進行兩次線性差值來完成雙線性差值。而上述過程中，容易發生錯誤。但是透過這方法顯示的畫質比使用「最近鄰」好。

五、旋轉

- 程式碼

```
import cv2
import numpy as np
import math

img = cv2.imread('cat.jpg')
angle = float(input("輸入角度: "))
h, w, c = img.shape

#旋轉角度
theta = math.radians(angle)
# 中心點
cx = w/2
cy = h/2

rotated = np.zeros_like(img)

for new_y in range(h):
    for new_x in range(w):
        # 旋轉公式
        x = (new_x - cx) * math.cos(theta) + (new_y - cy) * math.sin(theta) + cx
        y = -(new_x - cx) * math.sin(theta) + (new_y - cy) * math.cos(theta) + cy

        # 邊界保護
        if 0 <= x < w and 0 <= y < h:
            rotated[new_y, new_x] = img[int(y), int(x)]

cv2.imshow('rotated Image', rotated)
cv2.imwrite('rotated.jpg', rotated) #旋轉
```

- 執行結果（旋轉 30 度）



- 心得討論

其效率極慢，程式碼過於複雜，需要手動實現並進行邊界檢查。而因為影像尺寸不變，所以導致旋轉後部分影像內容被截斷。

六、插入簽名檔

- 程式碼

```
import cv2
import numpy as np

# 讀入主圖
img = cv2.imread('cat.jpg')

# 讀入簽名圖 (保持 alpha 通道)
sign = cv2.imread('signed.png', cv2.IMREAD_UNCHANGED)

# 假設簽名圖是 RGBA
if sign.shape[2] == 4:
    b, g, r, a = cv2.split(sign)
    sign_rgb = cv2.merge((b, g, r))
    alpha = a
else:
    # 若沒有透明通道，就直接用 RGB
    sign_rgb = sign
    alpha = np.ones(sign_rgb.shape[:2], dtype=np.uint8) * 255

# 縮放簽名圖
max_width = img.shape[1] // 5
scale = max_width / sign_rgb.shape[1]
new_w = int(sign_rgb.shape[1] * scale)
new_h = int(sign_rgb.shape[0] * scale)
sign_rgb = cv2.resize(sign_rgb, (new_w, new_h))
alpha = cv2.resize(alpha, (new_w, new_h))

# 計算右下角位置
h, w = img.shape[:2]
x_offset = w - new_w - 10
y_offset = h - new_h - 10

# ===== 手動像素判斷疊合 =====
for y in range(new_h):
    for x in range(new_w):
        # 若 alpha 非 0 (代表該點有簽名)
        if alpha[y, x] > 0:
            img[y + y_offset, x + x_offset] = sign_rgb[y, x]

# =====

cv2.imshow("Signed", img)
cv2.imwrite("signed.jpg", img)
cv2.waitKey(0)
```

- 執行結果



- 心得討論

相較於內建函式，這段程式碼僅進行簡單2值判斷(是否 > 0)，不適合實際運用。

七、比較最近鄰和線性

1. 影像品質

「線性」優於「最近鄰」。

「線性」新像素的值是根據原始影像周圍4個像素值，再依據距離加權平均計算出，因此線條會較為平滑。而「最近鄰」則是直接將浮點數座標取整數導致影像中的線條會出現鋸齒狀。

2. 計算複雜度

「最近鄰」較為簡單。僅需取整數。而「線性」則是需進行浮點數乘法和加法運算。

3. 執行速度

「最近鄰」為最快的插值方式。「線性」比「最近鄰」但是通常能在速度和畫質間取得平衡。